

PROYECTO POO

SISTEMA DE GESTIÓN DE INVENTARIO

Alumno:

Alejandro Terzano

Profesor:

Hernán Loisa

Profesor:

**Programacion Orientada a
Objetos**

Descripción del sistema:

¿Qué problema resuelve?

El sistema fue diseñado con la idea de mantener un inventario de un centro médico, pudiendo archivar insumos, proveedores, movimientos y empleados. Entre los insumos que se pueden manipular se cuentan Insumos médicos, administrativos y de limpieza, cada uno con sus propios atributos pensados para mayor especificación entre productos y un control detallado.

¿Quiénes son los usuarios?

Cuenta con dos tipos de usuarios, Administradores y Empleados, originalmente se tenía planeado dividir los permisos en base a estos roles, pero por complicaciones de tiempo y desarrollo se decidió que ambos roles tuvieran los mismos permisos manteniendo la distinción de cada uno en los registros con tal de que si algún movimiento es realizado por alguien que no tenía autorización para ello se le pueda rastrear en base a los registros.

¿Qué funcionalidades ofrece el sistema?

Gestión de insumos

- Alta de insumos
- Baja lógica
- Reactivación
- Consulta de información
- Control de stock mínimo
- Clasificación por tipo

Gestión de proveedores

- Alta de proveedores
- Baja lógica
- Reactivación
- Consulta de datos

Gestión de Movimientos

- Registro de entradas
- Registro de salidas
- Historiales completos de movimientos

Gestión de reportes

- Reporte de empleados
- Reporte de proveedores
- Reporte de insumos
- Reporte de movimientos
- Impresión de reportes

Persistencia

- Guardado automático en archivos CSV
- Recuperación de datos al iniciar el sistema

Decisiones del sistema:

Jerarquía de clases

La clase dominante en el sistema es “Inventario”, esta es el núcleo del sistema y centraliza la gestión de todas las entidades. Esta clase mantiene las colecciones, coordina las operaciones, la autenticación de usuarios, registra los movimientos y genera los reportes.

Elegí este diseño para evitar la dispersión de responsabilidades entre varias clases y a la vez facilitar el mantenimiento y orden del sistema, estando unificado de este modo se pueden encontrar las variables mas rápido, se evita tener que importar clases constantemente y se evita la posibilidad de que algún error en alguna clase que se necesite para compilar acciones mas grandes cause un problema en todo el sistema a la vez.

Relaciones del sistema

(Se agruparán aquellas clases con multiplicidades o relaciones iguales por simple conveniencia y para no repetir demasiadas veces las mismas justificaciones)

Insumo {abstracta} ----- Herencia ----- InsumoAdministrativo

Insumo {abstracta} ----- Herencia ----- InsumoLimpieza

Insumo {abstracta} ----- Herencia ----- InsumoMedico

decidí crear la clase abstracta de insumo para mantener los datos en común de todos los tipos de insumos para luego heredarlo a las clases hijas y que estas aporten sus propios atributos y métodos.

Inventario ---- Insumo (Multiplicidad: Inventario – 0..* Insumo)

Inventario ---- Proveedor (Multiplicidad: Inventario – 0..* Proveedor)

Inventario ---- Empleado (Multiplicidad: Inventario – 0..* Empleado)

Inventario ---- Movimiento (Multiplicidad: Inventario – 0..* Movimiento)

Se utiliza composición ya que el inventario se encarga de administrar y almacenar todos los elementos del sistema que corresponden a estas clases (todos los insumos, todos los proveedores, todos los empleados y todos los movimientos).

Proveedor ---- Insumo (Multiplicidad: Proveedor 1 ----- 0..* Insumos)

Cada proveedor puede suministrar múltiples insumos, pero cada insumo puede pertenecer a un solo proveedor.

Empleado ---- Movimiento (Multiplicidad: Empleado 1 ---- 0..* Movimiento)

Cada Empleado puede registrar múltiples movimientos, pero cada movimiento es realizado por un solo empleado.

Insumo ----- Movimiento (Multiplicidad: Insumo 1 ---- 0..* Movimiento)

Un insumo puede aparecer en múltiples movimientos, pero cada movimiento puede estar ligado solo a un insumo.

{Reportable} ----- Empleado/Proveedor/Insumo/Movimiento

Implemente la interfaz con el objetivo de unificar los reportes, gracias a esto se pueden generar todos los reportes de forma polimórfica usando solo este método común.

PersistenciaCSV ---- Empleado/Proveedor/Insumo/Movimiento

Esta clase es la encargada de almacenar y recuperar la información del sistema. Al iniciar la aplicación se cargan los datos previamente almacenados.

Conclusiones:

Dificultades encontradas

La realidad es que nunca había programado en java, por lo que la primera dificultad que encontré fue trabajar en un entorno completamente nuevo, si bien ya tenía algo de conocimiento en Python jamás había trabajado con objetos.

En la semana y media que pase desarrollando el programa también fue un gran problema lograr la estructura de clases que se pueda reutilizar, también generar todo el inventario como una clase “maestra” fue un desafío, tenía que crear todos los métodos que conectaran con el resto de las clases y que se comunicaran entre sí.

Tuve que corregir muchas veces el UML que realice al inicio junto con las multiplicidades, si deseara seguir con el proyecto estoy seguro de que tendría que hacer muchos cambios más.

Cambios que realizaría

Entre las cosas que quería cambiar (Pero ya sea por falta de tiempo o conocimientos) sería los distintos permisos por roles, Implementar filtros de búsqueda, agregar exportación de los reportes a PDF además de poder imprimirlos y edición de los movimientos registrados.

Sumado a eso también cambiaría el diseño, no lo pude cambiar por falta de tiempo y quedo con el estilo default proporcionado por swing.

Aprendizajes durante el desarrollo

Todo esto será desde mi perspectiva como único integrante de mi “grupo”.

Antes de empezar tenía 0 conocimientos sobre programar en java, por lo que llegar de ahí hasta lo que es la aplicación hoy es una curva enorme de aprendizaje y siendo sinceros no entendía mucho sobre la materia, pero solo hacia falta meterme de lleno en algo sobre esto y

mas por terquedad que otra cosa logre comprender todo lo que no había logrado comprender a lo largo del cuatrimestre.

Desde el Polimorfismo, pasando por la herencia, la persistencia de datos, el uso de swing y el diseño UML, creo que llegue a una buena comprensión de el entorno para lograr mejoras mas adelante y seguir programando sistemas similares.

Anexo

Diagrama UML

